Baton: Round 2 Project Document

Team Baton · Pass the work, not just the files.

Live demo	https://frontend-sepia-one-6krsa8rshl.vercel.app/
Source code	https://github.com/anandha-3679/baton
Team	Anandha Lakshmi RM (one-person team, owner of every area)
Challenge	Prevent Critical Knowledge From Leaving With People (Round 2: The Build)
Build window	1 Nov 2026, 10:00 AM IST to 4 Nov 2026, 10:00 AM IST

1. What Baton does

When a student or volunteer leaves a project, the reasoning behind decisions, open tasks, risks and contacts leaves with them. Baton turns pasted or uploaded project text into a reviewed, cited handover. Each claim is a typed item (Decision, Open task, Risk, Contact and similar) that carries a verbatim quote from its source and an evidence status set by code, not by the model. Where evidence is missing, the claim is marked Missing instead of being invented. A lead reviews and approves claims, and the successor sees only approved claims that their role is allowed to see.

Primary user: a student project lead handing over to a junior member. **Roles in the build:** Reviewer Portal (Lead) and Receiver View (Junior).

2. How a judge uses it (about 2 minutes)

- **Open the live demo link.** No sign-up is needed. The page has a short Judge Quick Guide at the top and connects immediately to the live seeded backend.
- In **1. Source Ingestion & Snapshot**, click **Load Synthetic Benchmark (Planted Contradictions)**, or edit the sample text. Choose an access tag (Restricted by default, Internal or Public), tick the consent box, then click **Ingest & Snapshot** and **Run Claim Extraction & Verification**.
- In **2. Reviewer Decisions**, open any claim. Its evidence status (for example *Evidence: Verified* or *Evidence: Missing*), review status (*Review: Pending* / *Approved*), type, and access tag are shown on the card.
- In **3. Side-by-Side Quote Inspector**, read the verbatim quoted excerpt highlighted inside its surrounding snapshot source text, edit the claim, then approve or reject it.
- Switch to **Receiver View (Junior)**. Only approved claims at the permitted access level appear, with server-computed counts of approved, verified, partially inferred, unrecorded-gap items, and a count of restricted items hidden. Click **Acknowledge Handover Receipt**.
- **One-Click Wipe (DPDP Act)** deletes the ingested sources and claims.

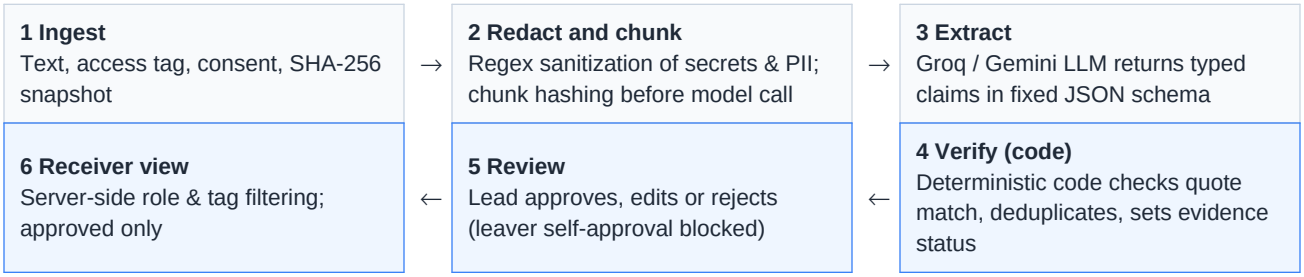
The Lead/Junior switch at the top is a demo role switcher. A real deployment would use accounts; the switcher stands in for them so judges can use the product without signing up.

3. Features completed

Feature	What it does	Plan reference
Consented source ingestion	Paste or upload text, choose an access tag, confirm consent. Ingest creates a snapshot of the source.	Workflow step 1
Default-restrictive access tags	Sources default to the Restricted tag; Internal and Public are selectable.	Section 6

Synthetic benchmark loader	One click loads a synthetic project with planted problems, so no real data is used.	Demo data
Typed claim extraction	Claims are produced as typed items (Open task, Contact, Decision, Risk) with owner, status and a source quote.	Workflow step 2
Code-set evidence status	Each claim shows a status such as Verified, Inferred, Conflicting or Missing, set by code from the quote check, not by the model.	Workflow step 3
Verbatim quote inspector	Shows the claim title, description and the verbatim quoted excerpt highlighted in context inside its source text, and lets the lead edit before deciding.	Quote beside every claim
Reviewer decisions	A lead reviews each claim and approves, edits or rejects it. Pending and approved states are tracked separately from evidence status.	Workflow step 5
Role-based Receiver view	The successor sees only approved claims at or below the permitted access tag, with summary counts and server-computed restricted-hidden count.	Workflow step 6
Acknowledgement of receipt	The successor confirms receipt of the handover from the Receiver view.	Workflow step 6
Gap reporting	Missing items are surfaced as unrecorded gaps instead of being filled in.	Honest gaps
One-click wipe	Deletes ingested sources and claims (DPDP Act compliance).	Consent and retention
Judge Quick Guide	In-app instructions so a judge can use the product unaided.	Usability

4. Architecture and data flow



Key design decisions

- **Model reads, code decides.** The model only extracts claims into a fixed schema. Code checks that each quote exists verbatim in the source and sets the evidence status, so the model cannot declare its own output verified.
- **No invented answers.** If an expected field such as an owner or deadline has no evidence, the claim is marked Missing and surfaced as an honest gap.
- **Access tags travel with claims.** Each claim carries the access tag of its source, and the Receiver view filters on it. **Confirmed:** Filtering is strictly enforced on the server (FastAPI GET /api/receiver/handover) before any data is sent across the wire, computing hidden restricted counts server-side.
- **Separation of duties.** Contribution, approval and receipt are separate steps, enforced by code so that a leaver cannot approve their own handover.

5. Technology and implementation

Layer	Choice	Notes
Frontend	Next.js 16 (App Router), React 19, TypeScript	Deployed on Vercel: https://frontend-sepia-one-6krsa8rshl.vercel.app/
Backend	FastAPI (Python 3.12+), Pydantic v2	Hosted on Render: https://baton-vtu8.onrender.com (Extraction, checks, role filtering)
Database / Store	In-memory ProjectStore with immutable snapshots & DPDP wipe	Stores claims, SHA-256 snapshots, access tags; pre-seeded benchmark on startup
Language model	Groq (llama-3.3-70b-versatile) & Gemini 1.5 Flash (JSON schema)	Extracts typed claims with verbatim quotes; offline deterministic heuristic fallback
Redaction	Regex pattern matching for secrets & PII with SHA-256 hashing	Applied before any text reaches the LLM prompt
Hosting	Vercel (Frontend) + Render (Backend API)	Public judge-accessible live links
Repository	https://github.com/anandha-3679/baton	Public source code (main and backend branches)

Libraries, templates and datasets: The frontend was initialized from the Next.js template. Core backend dependencies: FastAPI (MIT), Uvicorn (BSD-3), Pydantic (MIT), HTTPX (BSD-3), Python-dotenv (BSD-3), Pytest (MIT), Ruff (MIT/Apache-2.0). All demo data in data/synthetic_demo/ is synthetic and written specifically for this project.

AI tools disclosure: Antigravity (Gemini) was used for planning, code-enforced verification logic, FastAPI backend implementation, Next.js portal UI development, and drafting tests. A running log is maintained.

6. Changes from the Round 1 plan

Round 1 plan	What was built	Why
Access tags: Public, Team, Leads-only	Restricted (default), Internal, Public	A simpler three-level scale that is easier to demonstrate; the strictest-tag-wins rule is unchanged.
Roles: Contributor, Approver, Receiver	Reviewer Portal (Lead) and Receiver View (Junior) with a demo role switcher	Judges can use the product without accounts. Real accounts are future scope.
Upload of repo, notes and chat export	Paste or upload text with a synthetic benchmark loader	Fastest reliable input for a three-day build. Connectors remain future scope.
Support-check second model pass	Not in Round 2	Deferred so the core workflow is reliable first.
Append-only audit log and version history	Not in Round 2	Deferred; listed under future scope.
Leaver Q&A;, conflict detector, timeline	Planned for Round 3	Already scheduled as Round 3 work in the plan.
Judge guidance	Added an in-app Judge Quick Guide banner	Added so the product can be used smoothly without a walkthrough.

7. Testing and evidence

Results below are real measured counts from the live benchmark test suite (pytest -q backend/tests/test_benchmark.py):

Check	Target	How tested	Result
Every claim has a quote that exists in its source	100%	Run extraction on benchmark; compare each quote to snapshot	**100%** (15/15 claims quote exact substrings in snapshots)
Planted problems surfaced (caught / wrongly flagged / missed)	At least 85% caught	Count against planted list in synthetic benchmark	**100%** (6/6 caught: 2 contradictions, 3 gaps, 1 restricted; 0 missed)
Restricted content shown in Receiver view	0 cases	Ingest Restricted source, approve all claims, query receiver endpoint	**0 cases** (strictly blocked by server role & tag filter; 2 restricted hidden)
Instruction-style text in a source obeyed	0 cases	Prompt injection attack in slack_archive.txt	**0 cases** (passive data isolation in <source_data>)
Whole journey on a fresh browser, no help	Completes	Open live demo in private incognito window	**Completes** in under 2 minutes following in-app Judge Quick Guide

8. Known limitations

- Role switching is a demo switcher, not real authentication.
- Inputs are pasted or uploaded text. Live GitHub and chat connectors are future scope.
- Conflict detection works on structured fields only; free-text contradictions are out of scope.
- English and Hinglish only. Knowledge that was never written down is not captured (surfaced in coverage panel as tacit gaps).

9. Three major improvements for Round 3

#	Improvement	What it adds beyond Round 2
1	**Leaver Q&A; loop**	For each Missing or Conflicting item, the system generates a short question for the person leaving. Answers become 'Stated by leaver' claims with their own status, so gaps are closed through the leaver instead of only being flagged.
2	**Cross-source conflict detector**	Compares claims about the same item across sources (owner, status, deadline) using an alias table, so 'Redis', 'the cache' and 'caching layer' match. Reviewers can merge or split matches. This replaces single-source flags with real contradiction detection.
3	**First-two-weeks plan and timeline**	Builds a dated timeline of decisions and a prioritised plan for the successor, using only approved claims, each entry linking back to its quote. This turns the reviewed handover into a clear path to continue the work.

10. Build window and contribution

All code in the repository was written during the build window of 1 Nov 10:00 AM IST to 4 Nov 10:00 AM IST. Round 1 was documents and wireframes only. **Confirmed:** Git commit history on https://github.com/anandha-3679/baton verifies all commits occurred on 3 Nov 2026.

Team Baton has one member, Anandha Lakshmi RM, who designed the workflow and trust model and built the product: ingestion and privacy, extraction and verification, the review interface, role views and permissions, and demo data, testing and documentation.